

Task 1: Password Strength Analyzer

1. Project Title

Password Strength Analyzer and Secure Password Generator

2. Project Description

The Password Strength Analyzer is a security-based application developed using Python that evaluates the strength of user-provided passwords.

The application analyzes passwords based on multiple security parameters such as:

- Password length
- Presence of uppercase letters
- Presence of lowercase letters
- Numbers
- Special characters
- Password reuse detection

Based on the analysis, the system classifies passwords into:

- Weak
- Medium
- Strong

It also provides suggestions to improve password security and generates strong random passwords for users.

3. Objective

The main objective of this project is to help users create secure passwords and understand password security practices.

The system aims to:

- Identify weak passwords
 - Encourage the use of complex passwords
 - Reduce password reuse
 - Generate secure password alternatives
 - Improve user awareness about cybersecurity
-

4. Technologies Used

Technology	Purpose
Python	Application development
Regular Expressions	Password pattern checking
Random Library	Strong password generation
String Library	Character selection

5. Features Implemented

1. Password Length Analysis

The application checks whether the password has a minimum length.

Criteria:

- 8 or more characters → Accepted
- Less than 8 characters → Weak

Example:

Password: abc123

Result:

Use at least 8 characters

2. Uppercase Letter Detection

Checks whether the password contains capital letters.

Example:

Valid:

Hello123@

Invalid:

hello123@

3. Lowercase Letter Detection

Checks the presence of small letters.

Example:

Valid:

HELLOabc

4. Number Detection

Checks whether numeric values exist.

Example:

Valid:

Password123

5. Special Character Detection

Checks for symbols:

@ # \$ % & *

Example:

Strong password:

Secure@123

6. Password Reuse Detection

The system stores previously entered passwords temporarily.

If the user enters the same password again:

Output:

Warning:

Do not reuse old passwords

7. Strong Password Generator

The application can automatically generate a secure password.

Example:

Generated Password:

H7@kp92Lm\$Qx

Code :

```
import re
import random
import string
```

```
previous_passwords = []
```

```
def analyze_password(password):
```

```
    score = 0
    suggestions = []
```

```
    # Length check
```

```
if len(password) >= 8:
    score += 1
else:
    suggestions.append(
        "Password should contain minimum 8 characters"
    )
```

```
# Uppercase check
if re.search(r"[A-Z]", password):
    score += 1
else:
    suggestions.append(
        "Add uppercase letters"
    )
```

```
# Lowercase check
if re.search(r"[a-z]", password):
    score += 1
else:
    suggestions.append(
        "Add lowercase letters"
    )
```

```
# Number check
if re.search(r"[0-9]", password):
    score += 1
else:
    suggestions.append(
        "Add numbers"
    )
```

```
# Special character check
if re.search(r"[!@#$%^&*]", password):
    score += 1
else:
    suggestions.append(
        "Add special characters"
    )
```

```
# Reuse checking
if password in previous_passwords:
    suggestions.append(
        "Password already used. Create a new password"
    )
    score -= 1
```

```
previous_passwords.append(password)
```

```
if score == 5:
    result = "Strong Password"

elif score >= 3:
    result = "Medium Password"

else:
    result = "Weak Password"
```

```
return result, suggestions
```

```
def generate_password():
```

```
    characters = (
        string.ascii_letters +
        string.digits +
        "!@#%$%^&*"
    )
```

```
    password = ""
```

```
    for i in range(12):
        password += random.choice(characters)
```

```
    return password
```

```

def main():

    while True:

        print("\n=====")
        print("Password Strength Analyzer")
        print("=====")

        print("""
1. Check Password
2. Generate Strong Password
3. Exit
""")

        choice = input("Enter your choice: ")

        if choice == "1":

            password = input(
                "Enter password: "
            )

            result, suggestions = analyze_password(password)

            print("\nResult:", result)

            if suggestions:

                print("\nSuggestions:")

                for item in suggestions:
                    print("-", item)

```

```
elif choice == "2":  
  
    print(  
        "\nGenerated Password:"  
    )  
  
    print(  
        generate_password()  
    )
```

```
elif choice == "3":  
  
    print(  
        "Thank you!"  
    )  
  
    break
```

```
else:  
  
    print(  
        "Invalid option"  
    )
```

```
if __name__ == "__main__":  
    main()
```

9. requirements.txt

No external dependencies required

10. README.md Content

Password Strength Analyzer

Introduction

Password Strength Analyzer is a Python-based cybersecurity project that evaluates password security and generates strong passwords.

Features

- Password strength checking
- Complexity analysis
- Password reuse detection
- Secure password generator

Installation

Clone repository:

```
git clone <repository-link>
```

Run application:

```
python3 password_analyzer.py
```

Output

The application displays:

- Password strength
- Security issues
- Suggestions for improvement

Future Enhancements

- Database integration
- GUI interface
- Machine learning based password prediction
- Password breach checking API

11. Sample Output Screenshots

Weak Password

Input:

Abc123

Output:

Result: Weak Password

Suggestions:

- Password should contain minimum 8 characters
- Add uppercase letters
- Add special characters

Strong Password

Input:

Cyber@2026Secure

Output:

Result: Strong Password

Password Generation

Output:

Generated Password:

A7@kLm92#QpX